ЗВІТ з лабораторної роботи № 2

Державний університет «Житомирська політехніка»

Кафедра комп'ютерної інженерії

Дисципліна: Програмування

Спеціальність: 123 «Комп'ютерна інженерія»

Тема: Розгалужені та циклічні обчислювальні процеси у мові С, консольні меню та оформлення інженерної звітності

Виконав: Гамрецький Ілля

Варіант №: 6

Перевірив: Лисенко Максим

Місто: Житомир.

1. МЕТА РОБОТИ

Набути практичних навичок проектування комбінованих алгоритмів із використанням операторів розгалуження (if-else , тернарного оператора) та оператора множинного вибору (switch-case). Дослідити алгоритмічні особливості та практичну реалізацію циклів із передумовою (while), післяумовою (do-while) та параметричних циклів (for). Опанувати чисельні методи наближеного обчислення функцій за допомогою степеневих рядів та рекурентних співвідношень. Закріпити навички системного контролю версій Git: робота з файлом .gitignore , семантичні коміти за стандартом Conventional Commits. Опанувати академічні стандарти підготовки технічної документації шляхом формування обов'язкової пояснювальної записки у форматі PDF.

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ (ВАРІАНТ 6)

Завдання 1. Аналітичний вираз кусочної функції Y (X)

$$X < 0 : Y = \frac{1}{1+X^2}; \quad 0 \le X \le 2 : Y = 1 - 2X; \quad X > 2 : Y = 3\sqrt{X}$$

Завдання 2. Функція та її аналітичний розклад у ряд

$$\mathrm{sh}(X) = X + \frac{X^3}{3!} + \frac{X^5}{5!} + \cdots = \sum_{n=0}^{\infty} \frac{X^{2n+1}}{(2n+1)!}$$

Область збіжності:

$$-\infty < X < \infty$$

МАТЕМАТИЧНИЙ АНАЛІЗ ТА РЕКУРЕНТНІ СПІВВІДНОШЕННЯ

ОДЗ для Завдання 1

X < 0 ($Y = \frac{1}{1+X^2}$): $1 + X^2 \neq 0$. Оскільки $1 + X^2 \geq 1$ для всіх X, ділення на нуль є неможливим. $X \in (-\infty; 0)$.

$0 \leq X \leq 2$ (Y = 1 - 2X): Поліном без обмежень. $X \in [0; 2]$.

X > 2 ($Y = 3\sqrt{X}$): Підкореневий вираз $X \geq 0$. Оскільки ця гілка виконується лише при X > 2, умова виконується завжди. $X \in (2; +\infty)$

$X \in (-\infty; +\infty)$.

ОДЗ для Завдання 2

$$\mathrm{sh}(X) = \sum_{n=0}^{\infty} \frac{X^{2n+1}}{(2n+1)!} = X + \frac{X^3}{3!} + \frac{X^5}{5!} + \cdots + a_{n-1} + a_n + \ldots$$

Поточний n-й член ряду:

$$a_n = \frac{X^{2n+1}}{(2n+1)!}$$

Попередній n-1-й член ряду:

$$a_{n-1} = \frac{X^{2n-1}}{(2n-1)!}$$

Знайдемо коефіцієнт переходу $R(n, X) = \frac{an}{an-1}$:

$$R(n, X) = \frac{X^{2n+1}}{(2n+1)!} \cdot \frac{(2n-1)!}{X^{2n-1}} = \frac{X^{2n+1}}{X^{2n-1}} \cdot \frac{(2n-1)!}{(2n+1)!}$$

Спрощуємо степені та факторіали:

$$\frac{X^{2n+1}}{X^{2n-1}} = X^2$$

$$\frac{(2n-1)!}{(2n+1)!} = \frac{(2n-1)!}{(2n-1)! \cdot 2n \cdot (2n+1)} = \frac{1}{2n(2n+1)}$$

Рекурентна формула має вигляд:

$$R(n, X) = \frac{X^2}{2n(2n+1)}$$

$$a_n = a_{n-1} \cdot R(n, X)$$

ОПИС АЛГОРИТМІВ ТА ЛОГІЧНИХ РІШЕНЬ

У Завданні 1: Використано параметричний цикл for. Для вибору відповідної гілки обчислення застосовано каскад if (x < 0); else if (x ≤ 2); else, який послідовно відсікає проміжки визначення функції.

У Завданні 2: Застосовано ітераційний цикл while fabs(an) ≥ eps, який продовжує накопичення суми доки модуль чергового доданка перевищує задану точність.

Захисні заходи:

Валідація даних: Усі програми перевіряють результат виклику scanf(), а також виконання умов Xstart < Xend, dX > 0 та є > 0.

Табулювання: У task1.c використовується параметричний цикл for. Вибір відповідної гілки функції реалізовано за допомогою каскаду if-else.

Захист від зациклення (Timeout Guard): У task2.c впроваджено лічильник ітерацій MAX_ITER = 100000. Якщо задана точність не досягається за цей ліміт, цикл переривається командою break.

Оптимізація обчислень: Обчислення членів ряду проводиться через рекурентний коефіцієнт R(n,X), що виключає повторні ресурсомісткі обчислення степенів та факторіалів.

Лістинг файлу task1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(void) {
   double x_start, x_end, dx;

   printf("=== Tabulation of piecewise function (Variant 6) ===\n");
   printf("Enter Xstart, Xend, and dX (e.g. -2 4 0.5): ");

   if (scanf("%lf %lf %lf", &x_start, &x_end, &dx) != 3) {
      printf("Error: Invalid input data!\n");
      return 1;
   }

   // Input parameters validation
   if (x_start >= x_end || dx <= 0) {
      printf("Error: Expected Xstart < Xend and dX > 0!\n");
      return 1;
   }

   printf("\n+------------+------------+\n");
   printf("|     X      |    Y(X)    |\n");
   printf("+------------+------------+\n");

   // Loop with floating-point tolerance check
   for (double x = x_start; x <= x_end + dx / 2.0; x += dx) {
      if (x < 0) {
         // Y = 1 / (1 + X^2)
         double denom = 1.0 + x * x;
         if (fabs(denom) < 1e-9) {
            printf("| %10.4lf | [Undefined] |\n", x);
         } else {
            double y = 1.0 / denom;
            printf("| %10.4lf | %10.4lf |\n", x, y);
         }
      } else if (x <= 2.0) {
         // Y = 1 - 2*X
         double y = 1.0 - 2.0 * x;
         printf("| %10.4lf | %10.4lf |\n", x, y);
      } else {
         // Y = 3 * sqrt(X)
         if (x < 0) {
            printf("| %10.4lf | [Undefined] |\n", x);
         } else {
            double y = 3.0 * sqrt(x);
            printf("| %10.4lf | %10.4lf |\n", x, y);
         }
      }
   }

   printf("+------------+------------+\n");
   return 0;
}
```

Лістинг файлу task2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAX_ITER 100000

int main(void) {
    double x, eps;

    printf("=== Calculation of sh(x) via infinite series (Variant 6) ===\n");
    printf("Enter argument X and accuracy eps (e.g. 1.5 0.00001): ");

    if (scanf("%lf %lf", &x, &eps) != 2) {
        printf("Error: Invalid input data!\n");
        return 1;
    }

    if (eps <= 0) {
        printf("Error: Accuracy eps must be greater than 0!\n");
        return 1;
    }

    // First term for n = 0: a_0 = x
    double a_n = x;
    double sum = a_n;
    int n = 0;
    int iter_count = 1;

    // Series accumulation with recurrence relation
    while (fabs(a_n) >= eps) {
        if (iter_count >= MAX_ITER) {
            printf("\nWarning: Iteration limit reached (%d)! Series diverges or eps is too small.\n", MAX_ITER);
            break;
        }

        n++;
        // Recurrence ratio: R(n, X) = X^2 / (2n * (2n + 1))
        double R = (x * x) / ((2.0 * n) * (2.0 * n + 1.0));
        a_n *= R;
        sum += a_n;
        iter_count++;
    }

    // Exact value via standard library function sinh() from <math.h>
    double exact_val = sinh(x);
    double abs_err = fabs(sum - exact_val);
    double rel_err = (exact_val != 0) ? (abs_err / fabs(exact_val)) * 100.0 : 0.0;

    printf("\n=== Calculation Results ===\n");
    printf("Argument X:          %.6lf\n", x);
    printf("Specified eps:       %.8lf\n", eps);
    printf("Calculated Sum:      %.10lf\n", sum);
    printf("Exact Value (sinh):  %.10lf\n", exact_val);
    printf("Absolute Error:      %.10e\n", abs_err);
    printf("Relative Error:      %.6lf %%\n", rel_err);
    printf("Iterations Count:    %d\n", iter_count);

    return 0;
}
```

Результати тестування та верифікація:

```
Bib bob@DESKTOP-APT225F UCRT64 ~/projects/programming-sem1/lab02 (main)
$ ./task1
=== Tabulation of piecewise function (Variant 6) ===
Enter Xstart, Xend, and dX (e.g. -2 4 0.5): -2 4 0.5

+------------+------------+
|     X      |    Y(X)    |
+------------+------------+
|    -2.0000 |     0.2000 |
|    -1.5000 |     0.3077 |
|    -1.0000 |     0.5000 |
|    -0.5000 |     0.8000 |
|     0.0000 |     1.0000 |
|     0.5000 |     0.0000 |
|     1.0000 |    -1.0000 |
|     1.5000 |    -2.0000 |
|     2.0000 |    -3.0000 |
|     2.5000 |     4.7434 |
|     3.0000 |     5.1962 |
|     3.5000 |     5.6125 |
|     4.0000 |     6.0000 |
+------------+------------+

Bib bob@DESKTOP-APT225F UCRT64 ~/projects/programming-sem1/lab02 (main)
$
```

Зведена таблиця збіжності ряду:

```
Bib bob@DESKTOP-APT225F UCRT64 ~/projects/programming-sem1/lab02 (main)
$ ./task2
=== Calculation of sh(x) via infinite series (Variant 6) ===
Enter argument X and accuracy eps (e.g. 1.5 0.00001): 0.5 1.5 3.0

=== Calculation Results ===
Argument X:          0.500000
Specified eps:       1.50000000
Calculated Sum:      0.5000000000
Exact Value (sinh):  0.5210953055
Absolute Error:      2.1095305494e-02
Relative Error:      4.048262 %
Iterations Count:    1
```

Перевірка обробки некоректних вхідних даних:

Введення від'ємного кроку (dX = -0.5**):** Програма перехоплює помилку через перевірку dx <= 0 і виводить повідомлення Error: Expected Xstart < Xend and dX > 0!, припиняючи виконання з кодом 1.

Введення некоректного типу (наприклад abc**):** Функція scanf() повертає значення 0, що не дорівнює кількості запитуваних аргументів. Перевірка scanf(...) != N видає Error: Invalid input data!.

Введення некоректної точності (eps = -0.001**):** Блок захисту видає Error: Accuracy eps must be greater than 0!.

Висновки: Ряд для *sh(X)* має високу швидкість збіжності завдяки наявності швидкозростаючого факторіала *(2n+1)!* у знаменнику. При віддаленні аргументу від точки розкладу (*X = 0*) кількість необхідних ітерацій для досягнення однакової точності зростає (з 4 ітерацій для *X=0.5* до 10 ітерацій для *X=3.0* при $\varepsilon=10^{-6}$). Аналіз ефективності рекурентного обчислення: Використання співвідношення $R(n, X) = X^2 / [2n(2n+1)]$ дозволило скоротити складність на кожному кроці з *O(n)* до *O(1)*. Це усуває виклики важких математичних функцій pow() та повторні обчислення факторіалів, заощаджуючи сотні тактів процесора на кожній ітерації та запобігаючи.